INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

Department of Computer Science and Engineering

CSE-2340: Software Development 1

Group Idea-Based Web Development Project

Project detail	Requirement
Project type	Group project - exactly 3 students per group
Project selection	Each group proposes its own practical web application idea for instructor approval
Application scope	Frontend-focused web application using the course syllabus concepts
Page limit	Minimum 2 and maximum 4 pages/routes
Individual responsibility	Each member develops one distinct functional module; all modules are integrated into one application
Assessment	20 marks - individual module contribution and shared group deliverables

1. Project Description

Each group will design and develop an original, practical web application based on a problem or need selected by the group. The application may belong to any suitable domain, such as education, productivity, healthcare, business, personal management, community services or entertainment. The project must demonstrate the front-end development concepts and tools covered in CSE-2340: Software Development 1.

The project is completed by exactly three students. Each student must take ownership of a different, clearly defined functional module. The modules must work together as parts of one coherent application; they must not be three unrelated mini-projects. The project is frontend-focused and does not require a custom backend.

2. Group Structure and Individual Module Responsibilities

Before development begins, the group must divide the application into three distinct functional modules and assign one module to each member. Each member is responsible for implementing their assigned module, including its interface, interactions, validation and relevant data operations.

Member	Responsibility	Illustrative example: Online Learning Application
Member 1	Functional Module 1	Course Management - add, edit, delete and view courses
Member 2	Functional Module 2	Assignment Management - create assignments and track submissions
Member 3	Functional Module 3	Student Dashboard - show enrolled courses, progress and relevant summaries

The example is illustrative only. Groups may propose different application ideas and module divisions. A module may occupy a separate route, a section of a page, or a set of related components. The 2–4 page limit applies to the complete application, not to the number of modules.

Do not divide responsibilities only by technology (for example, one student doing HTML, another CSS and another JavaScript). Each member must develop a meaningful functional part of the application. All members should contribute to integration and maintain a clear record of their work in the shared GitHub repository.

3. Page and Navigation Requirements

The complete application must contain at least 2 and no more than 4 pages/routes. Use React Router for client-side navigation. A shared login/registration page may be used where authentication is needed. Forms, dialogs and modals do not need separate routes.

Possible page/route	Possible content	Access
1. Login / Registration	Email/password registration, sign-in and sign-out	Public
2. Main Dashboard	Overview, summaries, shortcuts or key information from the application	Protected where authentication is used
3. Module Workspace	One or more related functional modules, organized into sections or tabs	As appropriate
4. Additional Module / Profile	A remaining module, profile or supporting information	As appropriate; optional

4. Functional and Technical Requirements

- Application functionality: implement three distinct, useful modules that address the selected problem and integrate into one application.
- User interface: provide clear navigation, consistent styling, meaningful feedback and usable forms or controls.
- JavaScript: use variables, arrays, objects, functions, array methods and modern JavaScript features to implement application logic.
- React: use reusable components, props, state, effects, conditional rendering, lists, controlled forms and immutable state updates.
- Routing: use React Router for navigation between the application's pages/routes.
- Data handling: use appropriate client-side data structures and implement create, read, update or delete operations where relevant to the proposed application.
- Persistence: use Cloud Firestore for application records where persistent shared/user data is required. Use localStorage for a suitable lightweight preference or client-side setting.
- Authentication: implement Firebase Email/Password Authentication when the application has user-specific or protected functionality. Protect relevant routes and ensure users can access only their own records.
- API integration: use fetch() to retrieve and display relevant data from at least one suitable public API. Include loading and error handling; the API feature must not prevent the core application from working if the service is unavailable.
- UI component library: use at least one suitable ShadCN/ui component, such as Button, Card, Dialog or Table.
- Responsive design and accessibility: support desktop and mobile screen sizes and apply semantic HTML, accessible labels, keyboard navigation, visible focus and adequate contrast.
- Version control and release: use Git and GitHub, maintain meaningful commits, test core user flows, document the project in README and deploy the frontend.

5. Syllabus Coverage

Syllabus area	Expected application of concepts
Web fundamentals	Client/server architecture, HTTP request/response, status codes, browser rendering and hosting concepts, applied through the application and deployment.
Semantic HTML and accessibility	Document structure, headings, text, links, images, navigation, forms, accessible labels, landmarks, alternative text, heading order, keyboard navigation, focus visibility and adequate contrast.
CSS and layout	Selectors, specificity, cascade, text properties, units, box model, margin, padding, width, height, display, backgrounds, Flexbox, CSS Grid, positioning, z-index and responsive behavior.
Design handoff and	Interpret a simple Figma/Pixso-style design reference, spacing measurements and exported

Tailwind CSS	assets; apply Tailwind utility classes for typography, spacing, responsive layouts and interface styling.
JavaScript fundamentals	Variables, primitive values, template literals, arrays, objects and functions.
DOM and events	Selecting elements, working with content, attributes and classes, creating elements, event listeners and preventDefault; demonstrate these concepts in a small isolated interaction or explain their use where React manages rendering.
Array methods and modern JavaScript	map, filter, forEach, find, arrow functions, spread/rest, destructuring, optional chaining, object access, loops, truthy/falsy values and strict equality.
Asynchronous JavaScript and APIs	HTTP/HTTPS awareness, Promises, async/await, try/catch/finally and fetch(); process JSON responses and present loading, error and empty states. Use browser DevTools Network tab for debugging.
Browser persistence	Use localStorage for a suitable preference or lightweight client-side setting.
React with Vite	Project setup, JSX, reusable components, props, children, conditional rendering, lists, state, effects and immutable state updates.
React application structure	Lifting state up, controlled forms, form validation, component reuse and debugging common state/props issues.
React Router	Routes, Link/NavLink, Outlet, useNavigate, useParams where relevant, protected routes and a not-found page.
ShadCN/ui	Use at least one suitable component, such as Button, Card, Dialog or Table.
Firebase	Firebase SDK setup, Email/Password Authentication where user accounts are needed, sign-in/sign-out, authentication state, private routes and Firestore read/write operations where persistent application data is required.
Git, GitHub and release	Repository creation, .gitignore, meaningful commits, branches and pull requests where applicable; test core flows, debug with browser tools, check responsive behavior and accessibility, deploy and document setup/features in README.

6. Assessment Rubric - 20 Marks

Assessment criterion	Marks	Assessment focus
Individual Module 1 - implementation and functionality	4	Completeness, correctness, usability and the student's demonstrated ownership of the assigned module.
Individual Module 2 - implementation and functionality	4	Completeness, correctness, usability and the student's demonstrated ownership of the assigned module.
Individual Module 3 - implementation and functionality	4	Completeness, correctness, usability and the student's demonstrated ownership of the assigned module.
Integration and overall application functionality	3	Modules work together; navigation, shared data flow and main user workflows operate correctly.
Implementation of course concepts	2	Appropriate use of JavaScript, React, routing, forms, data handling and other relevant syllabus concepts.
Interface, responsiveness and accessibility	1	Clear interface, responsive behavior and basic accessibility practices.
GitHub, README and deployment	1	Organized repository, meaningful contribution history, project documentation and live deployment.
Testing and reliability	1	Core workflows tested; basic validation and error/empty states handled appropriately.
TOTAL	20	

The three individual module criteria account for 12 marks. The remaining 8 marks assess shared group outcomes. Individual marks may differ within a group based on each student's actual contribution and the quality of their assigned module.

7. Project Proposal and Submission Requirements

Before implementation, each group must submit a short proposal containing:

- Project title and a brief problem statement.
- Proposed solution and intended users.
- Three functional modules, with one named student responsible for each module.
- Main features and the proposed 2–4 pages/routes.
- A brief description of how the modules will share data or connect within the application.
- Planned use of the relevant technologies and syllabus concepts.

Final submission must include:

- GitHub repository URL containing the source code and contribution history.
- A zipped full project file submitted to the respective google form.
- Live deployed application URL.
- README containing the project description, features, module ownership, page/route information, setup instructions and Firebase configuration guidance where applicable.
- Screenshots of the completed application.
- Working authentication and data-persistence functionality where required by the approved project design.

8. Project Constraints and Notes

- Each group must contain exactly three students. The project is worth 20 marks.
- Each member must own one distinct functional module. The modules must be integrated into one application.
- The complete application must contain 2–4 pages/routes. A module does not necessarily require a separate page.
- The project is frontend-focused. A custom backend, server-side API or advanced database design is not required.
- The project idea and module division must be approved before implementation. The scope should be achievable using the course syllabus.
- Firebase Authentication and Firestore should be used where the approved application requires user accounts and persistent records. Apply suitable Firestore security rules when storing user-specific records.
- Do not commit service-account credentials or other privileged secrets to GitHub. Firebase client configuration must follow standard Firebase web-app practices.
- Use React state and event handling for React-rendered interface updates; do not manipulate React-managed DOM elements directly.
- The public API feature should be small and relevant. The core application must remain usable if the external API is unavailable.
- Students may customize the visual design. Assessment is based on implementation, individual contribution, integration and syllabus-aligned skills-not on how complicated or ambitious the project idea is.
- Tutorial-based copies or three unrelated modules presented as one project are not acceptable. Each group must demonstrate its own implementation and integration.